

Tema 6: Programación de bases de datos

Programación

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

Contenido

1. Introducción	3
2. Esquema general de procedimientos/funciones almacenadas	3
3. Hola Mundo	4
4. Comentarios	5
5. Procedimientos	5
5.1. Consulta procedimientos	7
6. Funciones	8
6.1. Consulta funciones	9
7. Variables	10
7.1. Declaración	10
7.2. Asignación de valor	10
7.3. Variables de usuario	12
8. Operadores	13
9. Sentencias de control	15
9.1. if	15
9.1. case	16
10. Instrucciones repetitivas	19
10.1. simple loop	19
10.2. while loop	21
10.3. repeat until loop	22
11. Modificación de datos	25
12. Consultas	27
13. Gestión de errores	36
13.1. Errores personalizados	40
14. Triggers	41

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

1. Introducción

En este tema se va a cubrir la programación de procedimientos/funciones almacenadas en un SGBD.

El lenguaje SQL ofrece una gran potencia para realizar consultas y gestionar datos en la base de datos, sin embargo existen acciones que no se pueden realizar directamente en SQL por lo que se necesita un lenguaje más potente. Oracle desarrolló un lenguaje procedimental para ampliar la potencia que ofrece SQL, el resultado fue el **PL/SQL (Procedural Language/Structured Query Language)**.

PL/SQL incluye las características propias de un lenguaje procedimental:

- El uso de variables.
- Estructuras de control de flujo y toma de decisiones.
- Control de excepciones.
- Reutilización del código a través de paquetes, procedimientos y funciones.

El código desarrollado en PL/SQL se puede almacenar como objetos de la base de datos, creando paquetes, funciones o procedimientos, pudiendo ser reutilizado por los usuarios que estén autorizados. Otra ventaja es que es ejecutado en el servidor, lo que supone un ahorro de recursos a los clientes.

Una parte muy importante de PL/SQL es la definición de disparadores (triggers), que permiten realizar una acción concreta sobre la base de datos cuando se ha modificado, insertado o borrado un registro de una tabla.

PL/SQL no es un estándar, pero la mayoría de SGBD lo han ido incorporando, aunque algunas como PostgreSQL han creado su propio lenguaje (PL/pgSQL, que es parecido a PL/SQL).

2. Esquema general de procedimientos/funciones almacenadas

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

Los procedimientos y funciones almacenadas en MySQL siempre siguen el siguiente esquema:

1. Nombre rutina + parámetros (entrada, salida, entrada/salida).
2. Declaración e inicialización de variables.
3. Procesamiento de datos: bloques `BEGIN`, `END` con instrucciones de control.
4. Fin, con instrucción `RETURN` en el caso de funciones.

En Oracle existen "bloques de código anónimo", es decir, se puede ejecutar directamente instrucciones de control sin definir una rutina, pero en MySQL no existe este concepto.

3. Hola Mundo

A continuación, se va a crear un procedimiento almacenado que muestre por pantalla "Hola Mundo":

Definir el procedimiento:

```
delimiter $$
```

```
drop procedure if exists holaMundo$$
```

```
create procedure holaMundo()
```

```
begin
```

```
select "hola mundo ";
```

```
end$$
```

Y ejecutar:

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
call holaMundo() $$
```

Consideraciones:

- `delimiter $$`: Esta palabra indica que carácter se usará para señalar el fin del script, es necesario definir uno ya que `;` se usa en PL/SQL para indicar las distintas instrucciones.
- `drop procedure if exists holaMundo` : muy importante siempre borrar el procedimiento que pudiera haber ya que si no dará error. En MySQL no se puede reemplazar procedimientos, siempre se tiene que borrar y crear.
- `create procedure holaMundo()` : Definición del procedimiento
- `begin/end $$`: para indicar comienzo y fin (es el equivalente a `{ }` de otros lenguajes como Java/C).
- `call holaMundo() $$`: para invocar el procedimiento.

Actividad 1

Crear un procedimiento almacenado que muestre la versión de MySQL:

4. Comentarios

Se pueden usar:

```
#
```

```
/* ... */
```

5. Procedimientos

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

Los procedimientos pueden tener argumentos de entrada, salida y entrada/salida, no devuelven ningún valor, su sintaxis es:

```
create procedure <nombre_procedimiento>
( IN|OUT|INOUT ARG1 <TIPO>,
  IN|OUT|INOUT ARG2 <TIPO>,
  ... )
BEGIN
<cuerpo procedimiento>
END
```

Por ejemplo, definir un procedimiento que reciba un argumento solo de entrada y lo muestre:

```
delimiter $$
drop procedure if exists mostrarUno $$
create procedure mostrarUno (
  IN ARG1 VARCHAR(50)
)
begin
select arg1;
end $$

delimiter ;

call mostrarUno('hola') ;
```

Procedimiento que recibe un argumento y lo multiplica por 2:

```
delimiter $$
drop procedure if exists multiplicarPor2 $$
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
create procedure multiplicarPor2 (
```

```
    INOUT ARG1 NUMERIC (3,0)
```

```
)
```

```
begin
```

```
set ARG1=ARG1*2;
```

```
end $$
```

```
delimiter ;
```

```
set @miVariable=5 ;
```

```
call multiplicarPor2 (@miVariable) ;
```

```
select @miVariable;
```

5.1. Consulta procedimientos

Mediante esta consulta se pueden obtener los procedimientos definidos:

```
show procedure status where db='ut6';
```

Para ver el contenido de un procedimiento se puede ejecutar:

```
show create procedure holaMundo;
```

Actividad 2

Crear un procedimiento almacenado que reciba como argumento un nombre y luego muestre "Hola " más el nombre.

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

6. Funciones

La principal diferencia entre un procedimiento y una función es que en una función los argumentos son solo de entrada y siempre tiene que devolver un valor. Su sintaxis es:

```
create function <nombre_funcion>
(ARG1 <TIPO>,
 ARG2 <TIPO>,
 ... )
RETURNS <TIPO>
BEGIN
<cuerpo función>
END
```

Para poder almacenar funciones en MySQL es necesario hacer:

```
SET GLOBAL log_bin_trust_function_creators=1;
```

Es posible que el usuario necesite el permiso "system_variables_admin" para poder realizar ese cambio, se otorgaría por ejemplo de la siguiente manera:

```
grant system_variables_admin on *.* to alumno@'localhost';
```

A continuación se muestra un ejemplo de función que devuelve el doble del valor recibido:

```
delimiter $$
drop function if exists doble $$
create function doble (
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```

    ARG1 NUMERIC (3,0)
)
RETURNS NUMERIC (4,0)
begin
declare resultado NUMERIC (4,0);
set resultado=ARG1*2;
return resultado;
end $$
delimiter ;

select doble(4);

```

6.1. Consulta funciones

Mediante esta consulta se pueden obtener las funciones definidas:

```
show function status where db='ut6';
```

Para ver el contenido de una función se puede ejecutar:

```
show create function doble;
```

Actividad 3

Crear un función almacenado que reciba como argumento un nombre y devuelva "Hola " más el nombre.

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

7. Variables

7.1. Declaración

PL/SQL incluye todos los tipos de datos de SQL e incluye alguno nuevo (por ejemplo `record`). Para declarar una variable se utiliza la siguiente sintaxis:

```
DECLARE <nombre> <tipo_dato> [DEFAULT valor]
```

Por ejemplo:

```
delimiter $$
drop procedure if exists procVariables $$
create procedure procVariables ()
begin
declare variable1 varchar(20);
declare variable2 varchar(20) DEFAULT 'hola';

end $$

delimiter ;
```

7.2. Asignación de valor

Se asignan con

```
set <variable> = <valor>;
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

También se puede asignar un valor ejecutando una consulta con `into`:

```
select column1, column2, ... into variable1, variable2, ...
...
```

Si la consulta devuelve más de un registro se producirá un error.

Por ejemplo:

```
create table usuarios (
    id_usuario int primary key,
    nombre varchar(50),
    activo boolean
);
```

```
insert into usuarios values
(1, 'Ana', TRUE),
(2, 'Luis', TRUE),
(3, 'Carlos', TRUE),
(4, 'Sofia', TRUE);
```

```
delimiter $$
drop procedure if exists procVariables $$
create procedure procVariables ()
begin
declare nuevoNombre varchar(50);

set nuevoNombre ='Paco';
select nombre into nuevoNombre
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```

from usuarios

where id_usuario=1;

select nuevoNombre;

end $$

delimiter ;

```

7.3. Variables de usuario

Las variables de usuario son aquellas que se definen en el entorno de MySQL, es decir, no son variables locales dentro de rutinas.

Estas variables no se declaran, por lo tanto no tienen tipo. Se les asigna valor de forma parecida a las variables locales teniendo en cuenta que hay que usar @ para referirse a ellas:

```

set @variable=2;

select columna into @variable

...

```

Por ejemplo:

```

set @variable=2; #asignar variable

select @variable; #mostrar variable

select nombre into @variable #asignar variable

from usuarios

```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
where id_usuario=1;
```

```
select @variable; #mostrar variable
```

```
delimiter $$
```

```
drop procedure if exists getNombre $$
```

```
create procedure getNombre (
```

```
out nombreOut varchar(50)
```

```
)
```

```
begin
```

```
select nombre into nombreOut
```

```
from usuarios
```

```
where id_usuario=2;
```

```
end $$
```

```
delimiter ;
```

```
call getNombre(@nombre);
```

```
select @nombre;
```

8. Operadores

Los operadores en PL/SQL son los mismos que en SQL, por ejemplo `+`, `-`, `=`, `!=`, `is null`, `between`, `and`, `not`, etc.

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

Actividad 4

Crear una función que reciba 2 argumentos (arg1 y arg2) y los devuelva en mayúsculas y concatenados con –, es decir, ARG1-ARG2.

Actividad 5

Dada la tabla:

```
create table pagos (
nombre varchar(100) not null primary key,
cantidad numeric (5,2) not null);
```

```
insert into pagos (
nombre,
cantidad)
values
('Luis', 200),
('Luisa', 400),
('Enrique', 100),
('Ana', 400),
('Mónica', 300);
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

Crear un procedimiento almacenado que reciba una cantidad y muestre los pagos iguales o mayores a esa cantidad.

Actividad 6

Con la tabla anterior crear un procedimiento que reciba un nombre y una cantidad y los inserte en la tabla.

9. Sentencias de control

9.1. if

Sintaxis es:

```
IF <condición> THEN
...
ELSEIF <condición> THEN
...
ELSE
...
END IF;
```

Por ejemplo:

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```

drop procedure if exists procif;

delimiter $$

create procedure procif (
    in valor int
)
begin

    if valor = 0 then

        select "valor es 0";

    elseif valor=1 then

        select "valor es 1";

    else

        select "valor no es ni 0 ni 1";

    end if;

end $$

delimiter ;

```

9.1. case

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

Sintaxis es:

```

CASE <expresion>
WHEN value THEN
...
WHEN value THEN
...
ELSE
...
END CASE;
```

Por ejemplo:

```

drop procedure if exists proccase;
delimiter $$
create procedure proccase (
    in valor int
)
begin
    case valor
        when 0 then
            select "valor es 0";
        when 1 then
            select "valor es 1";
        else
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```

select "valor no es ni 0 ni 1";

end case;

end $$

delimiter ;

```

Actividad 7

Realiza un procedimiento que reciba 2 números y muestre el mayor.

Actividad 8

Realiza un procedimiento almacenado que reciba un número entre 1 y 10 y muestre insuficiente si es menor que 5, suficiente entre 5 y 6, bien entre 6 y 7, notable entre 7 y 9 y sobresaliente entre 9 y 10. Cualquier otro número mostrará un mensaje de error.

Actividad 9

Realiza un procedimiento que reciba un número entre 1 y 7 y muestre el día de la semana (1- Lunes, 2-Martes, etc). Cualquier otro número mostrará un mensaje de error.

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

Actividad 10

Realiza un procedimiento que reciba dos parámetros que se corresponden con el resultado de un partido de fútbol. Deberá mostrar el resultado de la quiniela

Actividad 11

Realiza una función que reciba tres parámetros. Los dos primeros serían los operandos, el tercero la operación a realizar siendo 1 suma, 2 resta, 3 multiplicación y 4 división.

10. Instrucciones repetitivas

10.1. simple loop

Bucle infinito de sentencias, hay que programar específicamente la salida de él. **No es nada recomendable usarlo.** Su sintaxis es:

```
<etiqueta>:LOOP
instrucciones
END LOOP <etiqueta>;
```

Se puede salir del bucle añadiendo un `IF` para que cuando se cumpla una condición realice un `LEAVE <etiqueta>.`

Por ejemplo:

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```

drop procedure if exists procloop;

delimiter $$

create procedure procloop ()
begin

declare contador  INT;
declare resultado  varchar(200);

set contador =0;
set resultado = '';

etiqueta:loop

    set resultado = concat (resultado, contador , ', ');

    set contador = contador + 1;

    if contador=10 then

        leave etiqueta;
    end if;

end loop etiqueta;

select resultado;

end$$

```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

delimiter ;

10.2. while loop

Bucle que se ejecuta mientras se cumpla una condición. Su sintaxis es:

```
<etiqueta>:WHILE <expresion> DO
instrucciones
END WHILE <etiqueta>;
```

Por ejemplo:

```
drop procedure if exists procwhile;
delimiter $$
create procedure procwhile ()
begin

declare contador int;
declare resultado varchar(200);

set contador =0;
set resultado = '';

etiqueta:while contador < 10 do

    set resultado = concat (resultado, contador , ', ');

    set contador = contador + 1;
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
end while etiqueta;
```

```
select resultado;
```

```
end $$
```

```
delimiter ;
```

10.3. repeat until loop

Bucle que se ejecuta al menos una vez y **hasta** que se cumpla una condición (**no mientras** se cumpla una condición). Su sintaxis es:

```
<etiqueta>:REPEAT
instrucciones
UNTIL <expresion>
END REPEAT <etiqueta>;
```

Por ejemplo:

```
drop procedure if exists procrepeat;
delimiter $$
create procedure procrepeat ()
begin

declare contador int;
declare resultado varchar(200);

set contador =0;
set resultado = '';
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
etiqueta:repeat
```

```
    set resultado = concat (resultado, contador , ', ' );
```

```
    set contador = contador + 1;
```

```
until contador = 10
```

```
end repeat etiqueta;
```

```
select resultado;
```

```
end $$
```

```
delimiter ;
```

Actividad 12

Crea una función que reciba dos argumentos (límite inferior y límite superior), valide que ambos son positivos y que límite inferior es menor que límite superior. Si es correcto debe devolver 1, si no 0. Posteriormente crear un procedimiento almacenado que reciba esos dos argumentos (límite inferior y límite superior) y valide que son correctos usando la función. Si se supera la validación mostrar por pantalla los números pares entre ambos límites (inclusive).

Actividad 13

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

Crear un procedimiento almacenado que reciba dos argumentos (límite inferior y límite superior) y valide que son correctos usando la función del ejercicio anterior. Si se supera la validación mostrar por pantalla la suma de los números impares entre ambos límites (inclusive).

Actividad 14

Crear un procedimiento almacenado que muestre por pantalla las tablas de multiplicar del 0 al 9.

Actividad 15

Crea una función que determine si un número es primo (debe devolver 1 si lo es, 0 si no lo es).

Actividad 16

Dada la tabla:

```
create table numeros_primos (
numero numeric (5,2) not null primary key);
```

Crear un procedimiento almacenado que reciba dos argumentos (límite inferior y límite superior), valide que ambos son positivos y que límite inferior es menor que límite superior. Si se supera la validación deberá borrar los datos de la tabla y posteriormente insertar los

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

números primos entre ambos límites inclusive (usando la función de la actividad anterior). Finalmente debe realizar una consulta de la tabla.

11. Modificación de datos

Las sentencias insert, update y delete se pueden realizar como se realizan habitualmente, con la diferencia de que en lugar de usar datos constantes se pueden utilizar variables, por ejemplo:

```
drop procedure if exists procInsert;
delimiter $$
create procedure procInsert ()
begin

declare contador INT default 0;

drop table if exists tablaPrueba;

create table tablaPrueba (
id numeric(2,0) primary key,
descripcion varchar(50) not null
);

#Insertamos 10 filas
while contador < 10 do

insert into tablaPrueba (
id,
descripcion)
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```

values

(contador, concat ('registro ', contador));

set contador = contador + 1;

end while;

set contador = 5;

#Actualizamos la que tiene id 5

update tablaPrueba
set descripcion = concat ('registro nuevo ', contador)
where id=contador;

#Borramos la que tiene id 2

set contador = 2;

delete from tablaPrueba
where id=contador;

#Mostramos resultado

select * from tablaPrueba;

end$$

delimiter ;

```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

Mostrará:

```
+-----+-----+
| id | descripcion |
+-----+-----+
| 0 | registro 0 |
| 1 | registro 1 |
| 3 | registro 3 |
| 4 | registro 4 |
| 5 | registro nuevo 5 |
| 6 | registro 6 |
| 7 | registro 7 |
| 8 | registro 8 |
| 9 | registro 9 |
+-----+-----+
```

12. Consultas

A parte del `select .. into` se pueden hacer consultas que devuelvan más de una fila, para ello se utiliza un tipo de variable nuevo: el cursor (**se tiene que declarar después de otras variables**).

Para realizar una consulta se sigue el siguiente flujo:

1. Declarar el cursor:

```
DECLARE nombre_cursor CURSOR FOR sentencia_select;
```

2. Abrir el cursor:

```
OPEN nombre_cursor;
```

1. Extraer los datos de la fila actual:

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
FETCH nombre_cursor INTO variable1, variable2, ...;
```

Es muy importante nombrar las variables con nombres distintos a las columnas

1. Cerrar el cursor:

```
CLOSE nombre_cursor;
```

Para controlar cuando un cursor ha llegado al final de la consulta se debe utilizar la siguiente instrucción:

```
DECLARE nombre_variable BOOL;

DECLARE CONTINUE HANDLER FOR NOT FOUND SET nombre_variable = 1;
```

Por ejemplo:

```
drop procedure if exists procCursor;

delimiter $$

create procedure procCursor ()
begin

declare v_id numeric(2,0);
declare v_descripcion varchar(50);
declare fin_consulta BOOL default 0;

#declaracion cursor

declare miCursor CURSOR FOR

select id, descripcion

from tablaPrueba;

#declaración fin de consulta

DECLARE CONTINUE HANDLER FOR NOT FOUND SET fin_consulta = 1;

set fin_consulta=0;
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

#abrimos el cursor

OPEN miCursor;

loopcursor: LOOP

FETCH miCursor into v_id, v_descripcion;

if fin_consulta = 1 then

LEAVE loopcursor;

end if;

select concat ('Fila leida - ', v_id, ' - ', v_descripcion);

end loop loopcursor;

CLOSE miCursor;

end\$\$

delimiter ;

Y muestra:

```
+-----+
| concat ('Fila leida - ', v_id, ' - ', v_descripcion) |
+-----+
| Fila leida - 0 - registro 0                          |
+-----+
1 row in set (0.00 sec)
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
+-----+
| concat ('Fila leida - ', v_id, ' - ', v_descripcion) |
+-----+
| Fila leida - 1 - registro 1                          |
+-----+
1 row in set (0.04 sec)
```

```
+-----+
| concat ('Fila leida - ', v_id, ' - ', v_descripcion) |
+-----+
| Fila leida - 3 - registro 3                          |
+-----+
1 row in set (0.06 sec)
```

```
+-----+
| concat ('Fila leida - ', v_id, ' - ', v_descripcion) |
+-----+
| Fila leida - 4 - registro 4                          |
+-----+
1 row in set (0.09 sec)
```

```
+-----+
| concat ('Fila leida - ', v_id, ' - ', v_descripcion) |
+-----+
| Fila leida - 5 - registro nuevo 5                    |
+-----+
1 row in set (0.11 sec)
```

```
+-----+
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
| concat ('Fila leida - ', v_id, ' - ', v_descripcion) |
+-----+
| Fila leida - 6 - registro 6                        |
+-----+
1 row in set (0.14 sec)
```

```
+-----+
| concat ('Fila leida - ', v_id, ' - ', v_descripcion) |
+-----+
| Fila leida - 7 - registro 7                        |
+-----+
1 row in set (0.16 sec)
```

```
+-----+
| concat ('Fila leida - ', v_id, ' - ', v_descripcion) |
+-----+
| Fila leida - 8 - registro 8                        |
+-----+
1 row in set (0.18 sec)
```

```
+-----+
| concat ('Fila leida - ', v_id, ' - ', v_descripcion) |
+-----+
| Fila leida - 9 - registro 9                        |
+-----+
1 row in set (0.20 sec)
```

Actividad 17

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

Dada la tabla y datos:

```
create table empleados (
    id int primary key auto_increment,
    nombre varchar(100),
    salario decimal(10,2),
    departamento varchar(50)
);

insert into empleados (nombre, salario, departamento) values
('Ana', 1200, 'Ventas'),
('Luis', 1500, 'IT'),
('Marta', 1100, 'Ventas'),
('Carlos', 2000, 'IT'),
('Elena', 1300, 'Marketing');
```

Recorrer todos los empleados, si el empleado es de Ventas aumentar su salario un 10%, si es de IT aumentar su salario un 5%.

Actividad 18

Dada las siguientes tablas y datos:

```
create table clientes (
    id_cliente int primary key,
    nombre      varchar(50) not null,
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```

        saldo          numeric(10,2) not null
    );

create table productos (
    id_producto int primary key,
    nombre      varchar(50) not null,
    precio      numeric (5,2) not null,
    stock       int not null,
    constraint chk_stock check (stock >=0)
);

create table pedidos (
    id_pedido     int auto_increment primary key,
    id_cliente    int not null,
    fecha_hora    datetime not null,
    constraint fk_cliente foreign key (id_cliente)
        references clientes(id_cliente)
);

create table detalle_pedidos (
    id_pedido     int not null,
    id_producto   int not null,
    cantidad      int not null,
    constraint pk_detalle primary key (id_pedido, id_producto),
    constraint fk_pedido foreign key (id_pedido)
        references pedidos(id_pedido),
    constraint fk_producto foreign key (id_producto)
        references productos(id_producto)
);

```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
insert into clientes VALUES (1, 'Ana', 1000);
```

```
insert into clientes VALUES (2, 'Luis', 500);
```

```
insert into productos VALUES (1, 'Teclado', 50, 10);
```

```
insert into productos VALUES (2, 'Ratón', 10, 20);
```

```
insert into productos VALUES (3, 'Monitor', 200, 5);
```

Crear un procedimiento almacenado `procesar_pedido` que reciba los siguientes argumentos:

- `id_cliente: int IN`
- `id_producto: int IN`
- `cantidad: int IN`
- `resultadoOK: boolean OUT`
- `mensaje_error: varchar(100) OUT`

El procedimiento debe validar que hay suficiente stock y el cliente tiene suficiente saldo para la compra, si no se supera la validación `resultadoOK` será `FALSE` y `mensaje_error` contendrá una descripción del error. Si lo anterior es correcto se insertará datos en las tablas `pedidos` y `detalle_pedido` y se restará el stock y el saldo al cliente (**se debe garantizar la integridad en caso de que varias transacciones invoquen el procedimiento con igual cliente y/o producto**).

Actividad 19

Dada las siguientes tablas y datos:

```
create table clientes (
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```

        id_cliente int auto_increment primary key,
        nombre varchar(50) not null,
        saldo numeric(10,2) not null,
        constraint chk_saldo check (saldo >=0)

);

create table habitaciones (
        numero_habitacion numeric (3,0) primary key,
        precio_noche numeric(6,2)
);

create table reservas (
        id_reserva int auto_increment primary key,
        id_cliente int not null,
        numero_habitacion numeric (3,0) not null,
        fecha_entrada date,
        fecha_salida date,
        foreign key (id_cliente) references clientes(id_cliente),
        foreign key (numero_habitacion) references
habitaciones(numero_habitacion)
);

insert into clientes (nombre, saldo) values
('Carlos', 500),
('Marta', 150);

insert into habitaciones (numero_habitacion, precio_noche) values
(101, 100),
(201, 200),

```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

(301, 300);

Crear un procedimiento almacenado `gestionar_reserva` que reciba los siguientes argumentos:

- `id_cliente: int IN`
- `numero_habitacion: numeric (3,0) IN`
- `fecha_inicio: date IN`
- `fecha_fin: date IN`
- `resultadoOK: boolean OUT`
- `mensaje_error: varchar(100) OUT`

El procedimiento debe validar que las fechas son coherente y que el cliente tiene suficiente saldo para reservar la habitación, si no se supera la validación `resultadoOK` será FALSE y `mensaje_error` contendrá una descripción del error. Si lo anterior es correcto se insertará datos en la tabla `reservas` (se asume que existiría un trigger que no permitiría insertar reservas que en fechas ya reservadas) y se actualiza el saldo al cliente (**se debe garantizar la integridad en caso de que varias transacciones invoquen el procedimiento con igual cliente y/o habitación**).

13. Gestión de errores

Cuando un programa encuentra un error, la ejecución se detiene y se devuelve el error (comportamiento predeterminado). Pero podemos gestionar el error para mostrarlo y que el programa continúe su ejecución. Para ello se definen manejadores de errores. La sintaxis es:

```
DECLARE CONTINUE|EXIT HANDLER FOR <codigo_error_sql> <acciones>;
```

- `CONTINUE`: indica que no se corte el flujo
- `EXIT`: indica que finalice el bloque BEGIN-END actual
- `codigo_error`: el código que queremos capturar, también se puede usar `SQLXCEPTION` para calcular todos.

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

Por ejemplo, con la siguiente tabla definimos un procedimiento de inserción:

```
drop table if exists equipos;
drop procedure if exists insertar_equipo;
```

```
create table equipos (
    id int auto_increment primary key,
    equipo varchar(100) not null unique);
```

```
delimiter $$
```

```
create procedure insertar_equipo(
IN equipo varchar(100)
)
```

```
begin
    insert into equipos (
        equipo)
    values (
        equipo);
```

```
end$$
```

```
delimiter ;
```

Si se intenta insertar 2 veces el mismo equipo producirá un error:

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
mysql> call insertar_equipo ('Real Madrid');
```

```
Query OK, 1 row affected (0.02 sec)
```

```
mysql> call insertar_equipo ('Real Madrid');
```

```
ERROR 1062 (23000): Duplicate entry 'Real Madrid' for key 'equipos.equipo'
```

Por lo que podríamos capturar el error 1062 y gestionarlo mostrando un mensaje de error y permitiendo continuar.

```
drop procedure if exists insertar_equipo;
```

```
create table equipos (
    id int auto_increment primary key,
    equipo varchar(100) not null unique);
```

```
delimiter $$
```

```
create procedure insertar_equipo(
IN equipo varchar(100)
)
```

```
begin
```

```
    declare estado varchar(50) default null;
```

```
    declare continue handler for 1062 set estado='equipo ya existe';
```

```
    insert into equipos (
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```

equipo)

values (

equipo);

if estado is not null then

    select estado;

end if;

end$$

delimiter ;

```

Al ejecutar en lugar de mostrar el error por defecto se mostrará:

```
mysql> call insertar_equipo ('Real Madrid');
Query OK, 1 row affected (0.01 sec)
```

```
mysql> call insertar_equipo ('Real Madrid');
+-----+
| estado          |
+-----+
| equipo ya existe |
+-----+
1 row in set (0.01 sec)
```

```
Query OK, 0 rows affected (0.01 sec)
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

13.1. Errores personalizados

A parte de gestionar errores también es posible generar errores personalizados. La sintaxis es:

```
SIGNAL SQLSTATE '<codigo_error>'
    SET MESSAGE_TEXT = 'mensaje_de_error';
```

Por ejemplo, podemos tener un procedimiento almacenado que reciba un número que tenga que ser par y mayor que 0:

```
drop procedure if exists insertar_par;

delimiter $$

create procedure insertar_par
(IN numero numeric (3,0))
begin

    if numero <= 0 or numero mod 2 != 0 then

        signal sqlstate '45000'

        set message_text = 'El número tiene que ser par y mayor a 0';

    end if;

end $$

delimiter ;
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
call insertar_par (4);
call insertar_par (5);
```

El resultado será:

```
mysql> call insertar_par (4);
Query OK, 0 rows affected (0.00 sec)
```

```
mysql> call insertar_par (5);
ERROR 1644 (45000): El número tiene que ser par y mayor a 0
mysql>
```

14. Triggers

Un trigger es un código parecido a los procedimientos y funciones, pero que tiene la particularidad de estar asociado a una tabla y ejecutarse automáticamente como reacción a una operación DML específica (INSERT, UPDATE o DELETE) sobre dicha tabla.

La sintaxis es:

```
CREATE TRIGGER [IF NOT EXISTS] trigger_name
    trigger_time trigger_event
    ON nombre_tabla FOR EACH ROW
    [trigger_order]
    trigger_body
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

`trigger_time: { BEFORE | AFTER }`

`trigger_event: { INSERT | UPDATE | DELETE }`

`trigger_order: { FOLLOWS | PRECEDES } other_trigger_name`

Se puede programar el momento de la ejecución del `trigger` (`trigger_time`). Puede ser:

- `BEFORE`: indica que se ejecuta antes de la operación DML.
- `AFTER`: indica que se ejecuta después de la operación DML.

También se puede programar el tipo de sentencia DML que se desea que se dispare el `trigger` (`INSERT | UPDATE | DELETE`). **En MySQL no se puede usar el mismo trigger para 2 o mas sentencias DML (en otros SGBD sí), una solución es llamar a un procedimiento almacenado que realice la lógica de validación.**

Dentro del cuerpo del `trigger` se puede acceder a la información de la fila que se está procesando a través de dos pseudo-registros:

- `OLD`: Valores originales de la fila antes de la modificación.
- `NEW`: Valores nuevos a insertar/actualizar.

Al igual que con las funciones, para crear triggers en MySQL es necesario hacer:

```
SET GLOBAL log_bin_trust_function_creators=1;
```

Por ejemplo, se tiene la siguiente tabla de cuentas, si se desea que al intentar insertar una cantidad negativa se inserte en su lugar un 0 se puede programar un `trigger`:

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
drop trigger if exists trigger_insert;
drop table if exists cuenta;
```

```
create table cuenta (
numero_cuenta int primary key,
cantidad decimal (10,2));
```

```
delimiter $$
create trigger trigger_insert
before
insert
on
cuenta
for each row
begin
```

```
    if NEW.cantidad < 0 then
        set NEW.cantidad=0;
    end if;
```

```
end $$
```

```
delimiter ;
```

```
insert into cuenta (
numero_cuenta,
cantidad)
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

values

(1, -100);

select * from cuenta;

Se podría hacer un trigger que impida actualizar un saludo a un número negativo en la cuenta:

drop trigger if exists trigger_update;

delimiter \$\$

create trigger trigger_update

before

update

on

cuenta

for each row

begin

if NEW.cantidad < 0 then

signal sqlstate '45000'

set message_text = 'cantidad no puede ser negativo';

end if;

end \$\$

delimiter ;

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
update cuenta
set cantidad=-100;
```

Y el resultado sería:

```
mysql> update cuenta
-> set cantidad=-100;
```

ERROR 1644 (45000): cantidad no puede ser negativo

Y se puede observar que no se ha actualizado:

```
mysql> select * from cuenta;
+-----+-----+
| numero_cuenta | cantidad |
+-----+-----+
|          1   |    0.00  |
+-----+-----+
1 row in set (0.00 sec)
```

Actividad 20

Dada las siguientes tablas:

```
create table empleados (
    id int primary key,
    nombre varchar(100) not null,
```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```

        salario decimal(10,2) not null
    );

create table empleados_auditoria (
    id int auto_increment primary key,
    empleado_id int,
    salario_antiguo decimal(10,2),
    salario_nuevo decimal(10,2),
    fecha timestamp default current_timestamp ,
    constraint fk_empleados_auditoria foreign key (empleado_id) references
    empleados(id) on update cascade on delete cascade
);

```

Se desea llevar un histórico para posibles auditorías (`empleados_auditoria`) de todos los cambios de salario de un empleado. Crear un trigger para inserciones y otro para actualizaciones que valide que el nuevo salario es positivo (si no lo es lanzar un error) y luego inserte en la tabla `empleados_auditoria` si el salario cambia.

Actividad 21

Dada las siguientes tablas y datos:

```

create table clientes (
    id_cliente int auto_increment primary key,
    nombre varchar(50) not null,
    saldo numeric(10,2) not null,
    constraint chk_saldo check (saldo >=0)
);

```

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

```
);
```

```
create table habitaciones (
    numero_habitacion numeric (3,0) primary key,
    precio_noche numeric(6,2)
```

```
);
```

```
create table reservas (
    id_reserva int auto_increment primary key,
    id_cliente int not null,
    numero_habitacion numeric (3,0) not null,
    fecha_entrada date,
    fecha_salida date,
    foreign key (id_cliente) references clientes(id_cliente),
    foreign key (numero_habitacion) references
habitaciones(numero_habitacion)
```

```
);
```

```
insert into clientes (nombre, saldo) values
('Carlos', 500),
('Marta', 150);
```

```
insert into habitaciones (numero_habitacion, precio_noche) values
(101, 100),
(201, 200),
(301, 300);
```

Crear 2 triggers (uno para insert y otro para update) sobre la tabla reservas que compruebe que no exista otra reserva en la misma habitación con fechas que se solapen. Se considera que en la

IES Isidra de Guzmán	 IES Isidra de Guzmán
Bases de datos: 1º Desarrollo Aplicaciones Multiplataforma	Curso 2025/2026
Tema 6: Programación de bases de datos	Programación

fecha de salida puede comenzar otra reserva, es decir, por ejemplo la habitación 301 podría tener una reserva entre el 1-6-2026 y el 5-6-2026 y otra nueva reserva desde el 5-6-2026 al 8-6-2026, pero no podría tener una nueva reserva entre el 30-5-2026 y el 2-6-2026 o entre el 2-6-2026 y el 5-6-2026.